RELAZIONE PROGETTO RETI DI TELECOMUNICAZIONI
Di Tommaso Lana e Alessandro Rossi

Introduzione:

Il progetto si basa sulla realizzazione di un *Simulatore a eventi di un sistema a coda*. Il modello assegnatoci è quello di una coda M/M/c, ovvero una coda in cui:

- Gli arrivi sono determinati da un processo di Poisson di intensità **λ** e spostano il processo da uno stato i a i+1.
- Lo stato **i** è il **numero di clienti nel sistema**.
- I tempi di servizio per ciascun servitore seguono una distribuzione esponenziale di parametro **μ**.
- Il numero di servitori presenti nel sistema è **c**.
- Quando un utente termina il servizio e lascia il sistema, il processo passa dallo stato i allo stato i-1.
- Non c'è nessun limite al numero di utenti che possono essere contenuti nella coda FIFO.
- Vi è stabilità se $\rho = \lambda/(c\mu) < 1$.

Obiettivo:

Il nostro obiettivo è quindi quello di creare una simulazione in Java, che ci permetta di ottenere gli elementi caratteristici del sistema a coda, ovvero:

- La distribuzione **degli stati (Pk)**, cioè la frazione di tempo in cui nel sistema sono presenti esattamente k clienti.
- Il **numero medio di clienti nel sistema (Ls)** e il **numero medio di clienti nella coda (Lq)**.
- Il **tempo medio di permanenza nel sistema (Ws)** e il **tempo medio di permanenza nella coda (Wq)**.
- La **probabilità di attesa (**nel codice probAttesa**)**, ovvero la probabilità che un cliente trovi tutti i servitori occupati e debba attendere in coda.

Una volta ottenuti i parametri che ci interessano per verificare l'efficacia della simulazione verranno comparati con i dati teorici e costruiremo dei grafici con un programma scritto in Python.

Metodo:

Per costruire questo progetto abbiamo suddiviso lo schema generale di ciò che volevamo creare in singoli componenti, in modo da poterci suddividere il carico di lavoro e semplificare la realizzazione del tutto. Siamo partiti quindi con la definizione del modello teorico, dopo aver ripassato e raggruppato le formule necessarie siamo passati alla struttura del simulatore, e infine alla validazione dei dati ottenuti.

Simulatore:
Il simulatore è **a eventi discreti**, il tempo infatti non scorre linearmente ma avanza da un evento al successivo. I due possibili eventi sono ARRIVO e PARTENZA e facciamo uso di una lista degli eventi futuri (una PriorityQueue ordinata per tempo) che restituisce sempre l'evento più vicino.

L'esecuzione procede in quest'ordine: prendiamo il file config.json, lo carichiamo all'interno della classe Config, che ci permette di validare tutti i dati e interrompere la simulazione nel caso in cui ci sia una configurazione non valida.
Definiamo inoltre due liste per le classi RisultatiRun e RisultatiScenario, questo perché andremo a effettuare n simulazioni (il parametro *ripetizioni* nel config) per ogni coppia di valori (**λ, c**) e **μ** fisso, in pratica:

- Viene scelto lo scenario, ovvero i valori λ, μ e c.
- Vengono effettuate n simulazioni con questi parametri e ogni simulazione viene salvata nella lista di RisultatiRun.
- Una volta effettuate tutte le n simulazioni per scenario, sono salvate nella lista dei RisultatiScenario.

Così al termine dell'esecuzione il programma avrà tutti i risultati delle simulazioni suddivisi per scenario e ripetizione.

Dopodiché i dati ottenuti vengono salvati all'interno di un file chiamato risultati.json. Questo file viene letto da un programma chiamato plot.py che tramite la libreria matplotlib costruisce i relativi grafici comparando i dati simulati con quelli teorici.

Risultati e osservazioni:

Simulazione e teoria coincidono: la quasi totalità dei punti simulati cade sulle curve teoriche, entro l'intervallo di confidenza al 95%. Questo valida il simulatore; gli scostamenti dalle curve teoriche hanno una spiegazione *(vedi sezione "Dove la simulazione si scosta dalla teoria").*

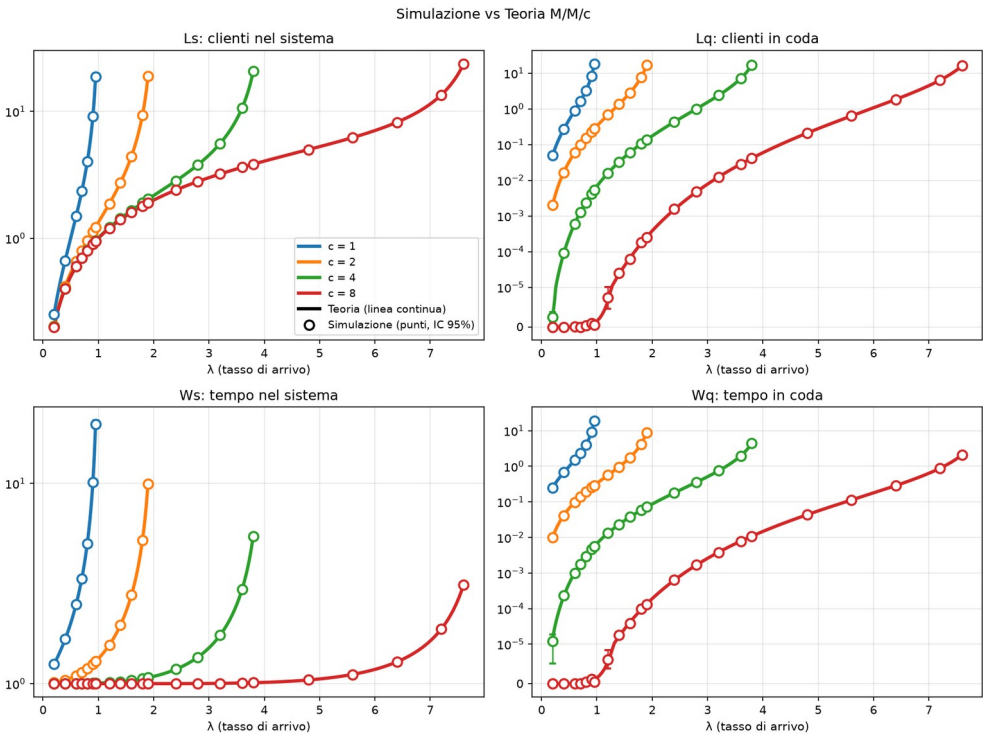
Setup dei grafici:

Nei grafici le linee sono la teoria, i punti la simulazione con IC 95% e l'asse y è logaritmico.
È stata scelta questa configurazione per i grafici finali:

```
"mu": 1,
"valoriC": [ 1, 2, 4, 8 ],
"valoriLambda": [ 0.2, 0.4, 0.6, 0.7, 0.8, 0.9, 0.95, 1.2, 1.4, 1.6, 1.8, 1.9, 2.4, 2.8,
3.2 , 3.6, 3.8, 4.8, 5.6, 6.4, 7.2, 7.6 ],
"tempoSimulazione": 200000,
"tempoWarmup": 20000,
"ripetizioni": 20,
"seme": 42,
"maxState": 150,
```

```
"outputPath": "risultati/risultati.json"
```

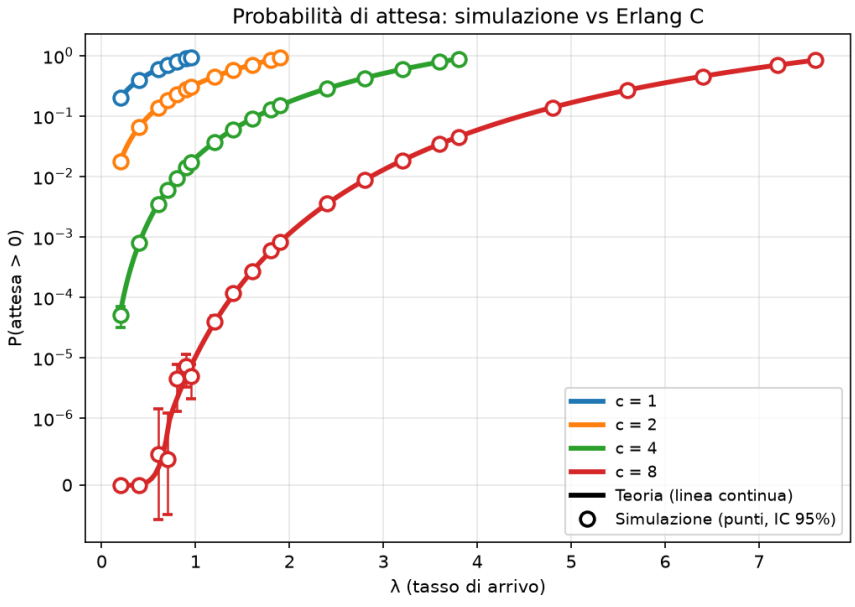
Ls, Lq, Ws, Wq:



- ***Crescita e divergenza***: Le metriche crescono lentamente a basso carico e molto rapidamente quando $\rho \to 1$. Le curve si fermano a $\rho=0.95$ perché per $\rho>=1$ la coda crescerebbe all'infinito, rendendo il sistema instabile (nel Main.java saltiamo direttamente le configurazioni instabili).
- ***Basso carico***: Se quasi nessuno aspetta, $Ls \approx \lambda/\mu$ qualunque sia c, quindi le curve si sovrappongono. Ws tende a $1/\mu$ (il solo tempo di servizio).
- ***Legge di Little***: Un'altra conferma della validità delle simulazioni è la correttezza di queste relazioni: $Lq = \lambda * Wq$ e $Ls = \lambda * Ws$. Per questo Lq e Wq (come Ls e Ws) hanno la stessa forma e differiscono solo per la scala. Valgono inoltre $Ls = Lq + (\lambda/\mu)$ e $Ws = Wq + 1/\mu$.

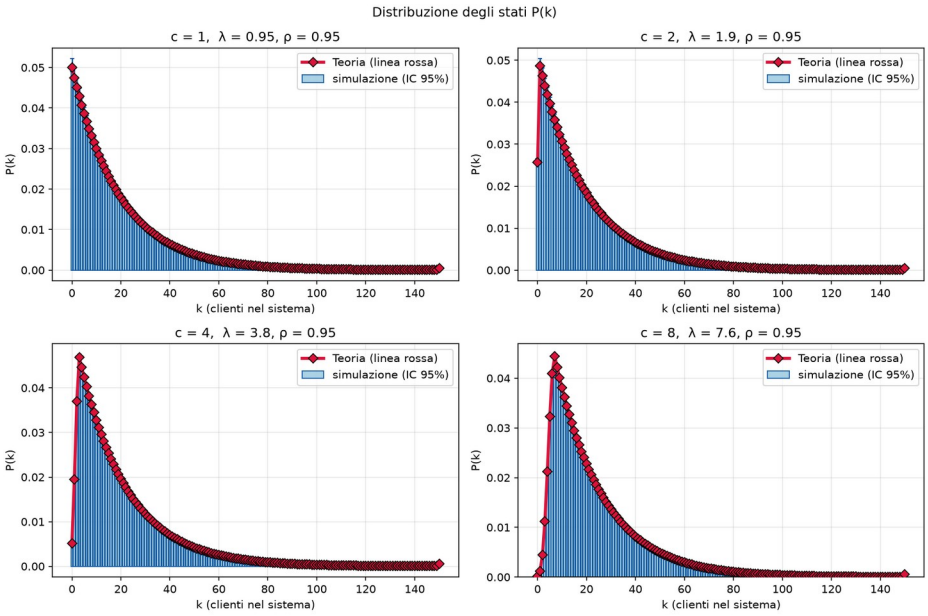
Probabilità di attesa:

La probabilità di attesa tende a 1 per $\rho \to 1$. Per c=1 vale esattamente ρ (qui $\rho = \lambda$ perché $\mu=1$) , anche se sull'asse logaritmico la curva non appare come una retta.



Distribuzione degli stati (ρ = 0.95):

- ***c = 1***: $P_k = (1 - \rho)\,\rho^k$: il massimo è in k=0 (P_0 = 0.05).
- ***c > 1***: P_k cresce finché $k < \lambda/\mu$, quindi il massimo è in k = floor(λ/μ), cioè k = c – 1. Da lì in poi decresce con la stessa pendenza per ogni c.
- ***Stato 0***: P_0 diventa quasi irraggiungibile al crescere di c.



Effetto del numero di servitori a parità di ρ:

Nei grafici le curve con c diverso sono confrontate a parità di λ, ma la capacità cresce con c e il confronto è quasi ovvio. A parità di ρ emerge l'effetto di condivisione della coda: più servitori su una coda unica fanno attendere molto meno. Ad alto carico quasi tutti attendono comunque, ma l'attesa si dimezza a ogni raddoppio di c. Ls invece cresce poiché con più servitori ci sono più clienti in servizio, anche se aspettano meno.

Dove la simulazione si scosta dalla teoria:

- ***Eventi rari***: Per c alto e λ piccolo la probabilità di attesa teorica è minuscola. La simulazione dà 0 con IC molto ampi, mentre la teoria dà valori piccolissimi ma non nulli. Ma questo è un limite della durata del run, non un errore del simulatore.
- ***Carico alto (ρ = 0.95)***: Gli IC sono più larghi perché la coda cambia lentamente e i campioni consecutivi sono correlati.

Conclusioni:

Il simulatore riproduce le prestazioni teoriche di un sistema M/M/c in tutti gli scenari considerati. Gli scostamenti osservati dipendono dalla rarità degli eventi e dal carico elevato, non dal modello.
Le simulazioni confermano due proprietà del sistema: tutte le metriche divergono per $\rho \to 1$ e a parità di ρ più servitori su un'unica coda riducono molto l'attesa media.